

Dispositivos Conectados

Barramento SPI: armazenamento em cartão microSD

2025/2026

Authors: Paulo C. Bartolomeu

Versão 1.1
7 de outubro de 2026

1 Objetivos

- Compreender o funcionamento do barramento SPI (sinais, *chip select* e modos de operação) e configurá-lo no ESP32-C6;
- Ler e escrever ficheiros num cartão microSD através do barramento SPI, usando o sistema de ficheiros FAT do ESP-IDF;
- Desenvolver uma aplicação de *logging* de temperatura e humidade (sensor I2C DHT20) para o cartão microSD, organizada em tarefas FreeRTOS.

2 Introdução

2.1 O barramento SPI

O SPI (*Serial Peripheral Interface*) é um barramento série síncrono e *full-duplex*, em que um controlador (o microcontrolador) comunica com um ou mais periféricos através de quatro sinais:

- **SCK** — relógio, gerado sempre pelo controlador;
- **MOSI** (também designado COPI, *Controller Out Peripheral In*) — dados do controlador para o periférico;
- **MISO** (também designado CIPO, *Controller In Peripheral Out*) — dados do periférico para o controlador;
- **CS** (*Chip Select*, ativo a nível baixo) — seleciona o periférico com que se pretende comunicar.

Vários periféricos podem partilhar SCK, MOSI e MISO, mas cada um tem a sua própria linha CS. Um periférico com o CS a nível alto ignora o barramento e mantém a sua saída MISO em alta impedância, o que permite que outro periférico a utilize. Em cada ciclo de relógio é transmitido um bit em cada sentido, pelo que uma leitura implica sempre o envio simultâneo de dados (ainda que irrelevantes).

O instante de amostragem dos dados é definido pelo **modo SPI**, que combina a polaridade do relógio em repouso (CPOL) com a fase de amostragem (CPHA):

Modo	CPOL (SCK em repouso)	CPHA (amostragem)
0	0 (nível baixo)	1.º flanco (ascendente)
1	0 (nível baixo)	2.º flanco (descendente)
2	1 (nível alto)	1.º flanco (descendente)
3	1 (nível alto)	2.º flanco (ascendente)

Controlador e periférico têm de usar o mesmo modo. Os cartões SD em modo SPI usam o modo 0.

No ESP32-C6, os controladores SPI0 e SPI1 estão reservados para o acesso à memória *flash*, sendo o **GP-SPI2** o controlador de uso geral disponível para a aplicação. Os seus sinais são encaminhados através da *GPIO matrix*, pelo que podem ser atribuídos a quase qualquer GPIO. Para frequências de relógio até 40 MHz, como as usadas nesta aula, o encaminhamento pela *GPIO matrix* tem o mesmo comportamento que a ligação direta pelo IO MUX.

2.2 Cartões SD em modo SPI

Os cartões SD suportam dois modos de comunicação: o modo SD nativo (1 ou 4 linhas de dados) e o modo SPI, mais lento mas suportado por praticamente qualquer microcontrolador. Após a alimentação, o controlador envia pelo menos 74 ciclos de relógio com o CS a nível alto e, de seguida, o comando `cmd0` com o CS a nível baixo, o que coloca o cartão em modo SPI. A inicialização decorre a uma frequência não superior a 400 kHz, podendo depois o relógio ser aumentado (o ESP-IDF usa 20 MHz por omissão). A comunicação é feita através de comandos de 6 bytes, a que o cartão responde com respostas curtas (p. ex. R1), e os dados são transferidos em blocos de 512 bytes.

2.3 Pilha de *software* no ESP-IDF

O acesso a um ficheiro no cartão atravessa várias camadas do ESP-IDF, o que permite à aplicação usar as funções padrão da biblioteca C (`fopen`, `fprintf`, `fgets`, ...):

Tabela 2: Camadas envolvidas no acesso a um ficheiro do cartão microSD.

Camada	Componente ESP-IDF	Função
Aplicação	<code>stdio.h</code> (<code>fopen</code> , <code>fprintf</code> , <code>fclose</code> , ...)	Operações sobre ficheiros
Sistema de ficheiros virtual	VFS	Associa um prefixo (<i>mount point</i> , p. ex. <code>/sdcard</code>) a um sistema de ficheiros
Sistema de ficheiros	FatFs	Organização FAT: diretórios, ficheiros, tabela de alocação
Protocolo SD	<code>sdmmc</code> (protocol layer)	Comandos SD, inicialização e leitura/escrita de setores

Camada	Componente ESP-IDF	Função
<i>Host</i> SD sobre SPI	<code>sdspi_host</code>	Traduz comandos SD em transações SPI
<i>Driver</i> SPI	<code>spi_master</code>	Controla o periférico GP-SPI2

A função `esp_vfs_fat_sdspi_mount()` inicializa e liga todas estas camadas numa única chamada.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência para os periféricos do ESP32-C6](#) juntamente com o [ESP32-C6 Technical Reference Manual](#). Adicionalmente, **deverão ser consultadas as fichas técnicas do sensor DHT20 e do *display* Adafruit 160x80 0.96"** disponíveis na página do elearning na pasta `Recursos`.

Versão do ESP-IDF: todos os exercícios devem ser realizados com o ESP-IDF **v6.1.0**. Noutras versões, o código do exemplo, a API I2C e as mensagens apresentadas na consola podem diferir.

Cartão microSD: utilize um cartão com capacidade **até 32 GB, formatado em FAT32** (o módulo suporta cartões em FAT16 ou FAT32). Cartões de maior capacidade (SDXC) vêm formatados em exFAT, que não está ativo por omissão no ESP-IDF. Faça uma cópia de segurança do conteúdo do cartão antes da aula.

3.1 Ligação do *display* com leitor de cartões integrado ao ESP32-C6 DevKit

O primeiro passo para a realização deste trabalho prático consiste na ligação elétrica do *display* ao ESP32-C6 DevKit. Como pode consultar na ficha técnica do *display*, este incorpora um leitor de cartões SD que será fundamental para esta aula prática. É possível visualizar o TFT e o leitor de cartões nas Figuras 1 e 2, estando disponível informação do *pinout* de ligação na camada *silkscreen* da PCB.

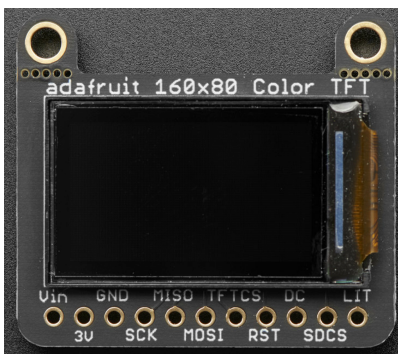


Figura 1: Display Adafruit (Vista do TFT)

O módulo partilha o mesmo barramento SPI entre o ecrã TFT e o leitor de cartões, cada um com o seu próprio *chip select*. O ecrã apenas recebe dados, pelo que o leitor de cartões necessita de duas linhas adicionais: `MISO` e `SDCS`. O módulo inclui um regulador LDO de 3,3 V (150 mA) e um

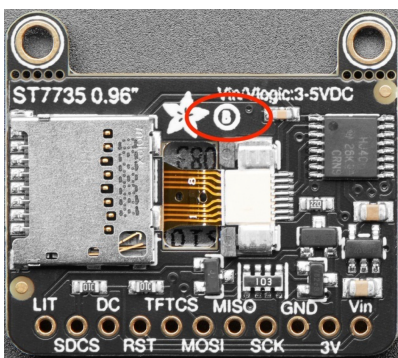


Figura 2: Display Adafruit (vista do leitor de cartões SD; o B dentro do círculo assinalado identifica a revisão B do módulo)

conversor de níveis CD74HC4050, com atraso de propagação típico de cerca de 10 ns, graças ao qual as entradas aceitam lógica de 3 a 5 V; a saída `MISO` é a 3,3 V. Neste laboratório iremos usar apenas o leitor de cartões, pelo que **o pino `TFTCS` deve ser ligado a 3,3 V de modo a que o ecrã permaneça desseleccionado durante todos os exercícios**. Os restantes sinais do ecrã (`RST`, `DC` e `LIT`) e a saída `3V` do regulador do módulo não são utilizados. Note que o pino `LIT` está a nível alto por omissão: deixado desligado, a retroiluminação do ecrã fica acesa e consome cerca de 25 mA. Para reduzir o consumo, pode ligar `LIT` a `GND`.

A Tabela 3 resume as ligações a efetuar, ilustradas na Figura 3. Note as **resistências de *pull-up* de 10 k Ω** nos sinais `MISO`, `MOSI` e `SDCS`. Proceda à montagem do circuito, tendo atenção redobrada às ligações efetuadas.

Tabela 3: Ligações entre o módulo Adafruit e o ESP32-C6 DevKit.

Pino do módulo	Função	Ligação ao ESP32-C6 DevKit
<code>Vin</code>	Alimentação (aceita 3–5 V)	<code>3V3</code>
<code>GND</code>	Massa	<code>GND</code>
<code>SCK</code>	Relógio SPI	<code>GPIO 21</code>
<code>MOSI</code>	Dados ESP32-C6 → cartão	<code>GPIO 19</code> (+ <i>pull-up</i> 10 k Ω)
<code>MISO</code>	Dados cartão → ESP32-C6	<code>GPIO 20</code> (+ <i>pull-up</i> 10 k Ω)
<code>SDCS</code>	<i>Chip select</i> do cartão SD (ativo baixo)	<code>GPIO 18</code> (+ <i>pull-up</i> 10 k Ω)
<code>TFTCS</code>	<i>Chip select</i> do ecrã TFT	<code>3V3</code> (ecrã sempre desseleccionado)
<code>3V</code> , <code>RST</code> , <code>DC</code> , <code>LIT</code>	Saída do regulador e sinais do ecrã	Não ligados

As resistências de *pull-up* não são exigidas pelo protocolo SPI, cujas saídas são *push-pull*, mas pelos cartões SD, que as requerem também em modo SPI ([SD Pull-up Requirements](#)). Garantem um nível definido nas linhas quando nenhum dispositivo as está a forçar: durante o arranque do ESP32, antes de os GPIO serem configurados, e na linha MISO sempre que o cartão está desseleccionado e a sua saída fica em alta impedância.

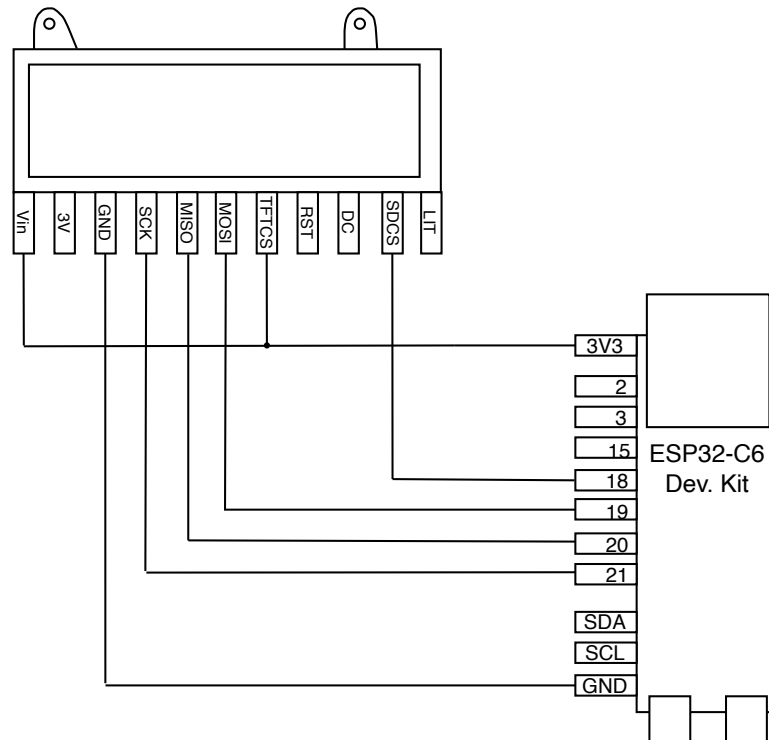


Figura 3: Circuito de ligação SPI ao leitor de cartões

Após a montagem do circuito insira o cartão microSD na ranhura do leitor.

3.2 Aplicação de teste para leitura/escrita de cartões SD

Através do *Wizard* da extensão ESP-IDF, crie um novo projeto tendo como base o exemplo `storage -> sd_card -> sdspi`.

Antes de compilar, abra o SDK Configuration Editor, procure a secção SD SPI Example Configuration e atribua os números dos pinos de acordo com o circuito montado:

Opção	Valor
MOSI GPIO number	19
MISO GPIO number	20
CLK GPIO number	21
CS GPIO number	18

ATENÇÃO: não ative nenhuma opção de formatação do cartão disponível nesta secção (p. ex. *Format the card if mount failed*). Se ativa, todo o conteúdo do cartão será apagado.

De seguida, compile o exemplo e descarregue-o para a placa. Deverá observar no terminal informação semelhante à seguinte (o nome, a capacidade e alguns detalhes do cartão variam de cartão para cartão):

```
I (444) main_task: Calling app_main()
I (444) example: Initializing SD card
I (444) example: Using SPI peripheral
I (454) example: Mounting filesystem
I (454) sdsapi_transaction: cmd=52, R1 response: command not supported
I (504) sdsapi_transaction: cmd=5, R1 response: command not supported
I (524) example: Filesystem mounted
Name: 00000
Type: SDHC
Speed: 20.00 MHz (limit: 20.00 MHz)
Size: 29820MB
CSD: ver=2, sector_size=512, capacity=61071360 read_bl_len=9
SSR: bus_width=1
I (524) example: Opening file /sdcard/hello.txt
I (544) example: File written
I (544) example: Renaming file /sdcard/hello.txt to /sdcard/foo.txt
I (554) example: Reading file /sdcard/foo.txt
I (554) example: Read from file: 'Hello 00000!'
I (554) example: Opening file /sdcard/nihao.txt
I (574) example: File written
I (574) example: Reading file /sdcard/nihao.txt
I (574) example: Read from file: 'Nihao 00000!'
I (574) example: Card unmounted
I (574) main_task: Returned from app_main()
```

Caso seja apresentado um erro, consulte a secção [Resolução de problemas](#) no final do guião.

QUESTÃO 1: Estude o *driver* SD ([SD SPI Host Driver](#)) e o suporte para o sistema de ficheiros ([FAT Filesystem Support](#)) e analise o código do exemplo.

- Identifique a sequência de passos realizados pelo exemplo e, para cada passo, a função da API utilizada (p. ex. `spi_bus_initialize()`, `esp_vfs_fat_sdspi_mount()`, `fopen()`, `rename()`, `esp_vfs_fat_sdcard_unmount()`, `spi_bus_free()`).
- Que controlador SPI é utilizado pelo exemplo? Consultando o *Technical Reference Manual*, quantos controladores SPI de uso geral possui o ESP32-C6?
- Explique o significado dos campos `format_if_mount_failed`, `max_files` e `allocation_unit_size` da estrutura `esp_vfs_fat_sdmmc_mount_config_t`.
- Qual o papel do *mount point* `/sdcard`? Porque é possível usar funções padrão como `fopen()` e `fprintf()` para aceder ao cartão?
- A consola indica uma velocidade de 20 MHz. Porque é que a inicialização do cartão não é feita diretamente a esta frequência?
- A que correspondem as mensagens `cmd=52` e `cmd=5 ... command not supported`? Porque não representam um erro?
- Porque deve o pino `TRFCS` ser ligado a 3,3 V? O que poderia acontecer se ficasse desligado? Para que servem as resistências de *pull-up* nos sinais `MISO`, `MOSI` e `SDCS`?

QUESTÃO 2: Desligue o ESP32-C6 DevKit da alimentação e retire o cartão microSD. Usando um leitor de cartões SD, inspecione o seu conteúdo.

- a) Que ficheiros criados pelo programa encontra e qual o seu conteúdo? Justifique, com base no código, a ausência do ficheiro `hello.txt`.
- b) Que data de modificação apresentam esses ficheiros? Explique porquê e indique como poderia o sistema registar a data e hora corretas.

3.3 Aplicação de *log* para escrita no cartão SD

Com base no exemplo anterior, pretende-se agora desenvolver variantes do programa com funcionalidades mais úteis. Em todos os programas seguintes:

- o cartão é montado em `/sdcard`;
- o valor de retorno de todas as funções da API e de `fopen()`/`fprintf()`/`fgets()` deve ser verificado, imprimindo-se uma mensagem de erro com `ESP_LOGE()` (no caso das funções de `stdio.h`, incluindo `strerror(errno)`);
- os nomes de ficheiro usados neste guião respeitam o formato 8.3 (até 8 caracteres de nome e 3 de extensão), o que garante o seu funcionamento mesmo que o suporte de nomes longos do FatFs esteja desativado (opção `CONFIG_FATFS_LFN_*`).

IMPORTANTE — remoção segura do cartão: os dados escritos com `fprintf()` ficam primeiro em memória (*buffers* da biblioteca C e do FatFs) e só são efetivamente gravados no cartão quando o ficheiro é fechado ou sincronizado. Retirar a alimentação ou o cartão com um ficheiro aberto pode provocar perda de dados ou corrupção do sistema de ficheiros. Por isso, nos programas de escrita, **cada registo deve ser seguido de `fclose()`** (ou de `fflush()` seguido de `fsync()`), e deve existir um mecanismo explícito para terminar o *logging* e desmontar o cartão antes de o remover. Em alternativa, a opção `CONFIG_FATFS_IMMEDIATE_FSYNC` faz com que o FatFs sincronize automaticamente o ficheiro após cada escrita.

QUESTÃO 3: Altere o programa anterior para registar no ficheiro `/sdcard/log.txt`, em formato CSV, o tempo decorrido desde o arranque da placa e um valor de contagem. O programa deve cumprir os seguintes requisitos:

- o registo é realizado por uma tarefa FreeRTOS com um período de **1000 ms**, garantido com `vTaskDelayUntil()`;
- o tempo em milissegundos é obtido com `esp_timer_get_time() / 1000`; o contador começa em 0 e é incrementado a cada registo;
- formato: a primeira linha é o cabeçalho `millis,counter`; cada linha seguinte contém os dois valores inteiros separados por vírgula, sem espaços, terminada com `\n`;
- se o ficheiro **não existir** no arranque (verifique com `stat()`), é criado e o cabeçalho é escrito; se **já existir**, os novos registos são acrescentados no fim (modo "a") sem repetir o cabeçalho — assim, cada novo arranque da placa inicia uma nova sequência com `millis` e `counter` a recomençar;
- o ficheiro é aberto, escrito e fechado a cada registo;
- ao premir o botão `BOOT` (GPIO 9, ativo a nível baixo — configure o *pull-up* interno), a tarefa termina o *logging*, desmonta o cartão, liberta o barramento SPI, imprime a mensagem «Cartão desmontado. Pode remover o cartão.» e termina.

- a) Implemente o programa, execute-o durante alguns segundos, termine o *logging* com o botão `BOOT`, desligue a placa e leia o ficheiro `log.txt` no portátil.
- b) Numa primeira versão de uma solução, que usava `vTaskDelay(pdMS_TO_TICKS(1000))`, obteve-se a sequência de tempos 270, 1340, 2400, 3460, Explique porque é que o intervalo entre registos excede 1000 ms nessa versão e porque é que `vTaskDelayUntil()` resolve o problema.

O ficheiro `log.txt` deverá ter um conteúdo semelhante ao seguinte:

```
millis,counter
271,0
1271,1
2271,2
3272,3
4271,4
5271,5
6271,6
```

3.4 Aplicação de leitura do *log* escrito no cartão SD

Pretende-se agora desenvolver um programa que, ao invés de escrever informação para o ficheiro de *log*, leia essa informação linha a linha e a imprima na consola.

QUESTÃO 4: Desenvolva um programa em que uma tarefa FreeRTOS leia, a cada segundo, uma linha do ficheiro `/sdcard/log.txt` gerado na questão anterior e a imprima no terminal. O programa deve cumprir os seguintes requisitos:

- o ficheiro é aberto em modo de leitura uma única vez, no início da tarefa, e as linhas são lidas com `fgets()` para um *buffer* de 64 bytes;
- **a primeira linha (cabeçalho) é ignorada**, bem como linhas vazias;
- o carácter de fim de linha é removido antes de imprimir; cada linha é impressa no formato CSV: `<linha>`;
- se o ficheiro não existir, é impressa uma mensagem de erro e a tarefa termina;
- ao atingir o fim do ficheiro, é impresso EOF: `<n> linhas lidas` (sem contar o cabeçalho), o ficheiro é fechado, o cartão é desmontado e a tarefa termina (`vTaskDelete(NULL)`).

O resultado deverá ser semelhante ao apresentado em seguida (para o ficheiro da questão anterior):

```
I (446) main_task: Calling app_main()
I (446) sdread: Mounting /sdcard
I (456) sdsapi_transaction: cmd=52, R1 response: command not supported
I (496) sdsapi_transaction: cmd=5, R1 response: command not supported
I (516) sdread: Mounted. Card: 00000
I (516) main_task: Returned from app_main()
I (516) sdread: CSV: 271,0
I (1516) sdread: CSV: 1271,1
I (2516) sdread: CSV: 2271,2
I (3516) sdread: CSV: 3272,3
I (4516) sdread: CSV: 4271,4
I (5516) sdread: CSV: 5271,5
I (6516) sdread: CSV: 6271,6
I (7516) sdread: EOF: 7 linhas lidas
I (7526) sdread: Card unmounted
```

3.5 Integração da leitura da temperatura e humidade (DHT20) com *logging* no cartão microSD

Finalmente, pretende-se integrar as funcionalidades anteriores com o sensor de temperatura e humidade DHT20 (I2C, endereço 0x38) usado na aula anterior. Para tal, deverá montar o circuito da Figura 4, que acrescenta ao circuito anterior o sensor DHT20 com as respetivas resistências de *pull-up* de 4,7 k Ω nas linhas SDA e SCL. As ligações do sensor são as seguintes:

Tabela 5: Ligações entre o sensor DHT20 e o ESP32-C6 DevKit.

Pino do DHT20	Função	Ligação ao ESP32-C6 DevKit
1	VDD	3V3 (+ condensador 100 nF para GND)
2	SDA	GPIO 6 (+ <i>pull-up</i> 4,7 k Ω)
3	GND	GND
4	SCL	GPIO 7 (+ <i>pull-up</i> 4,7 k Ω)

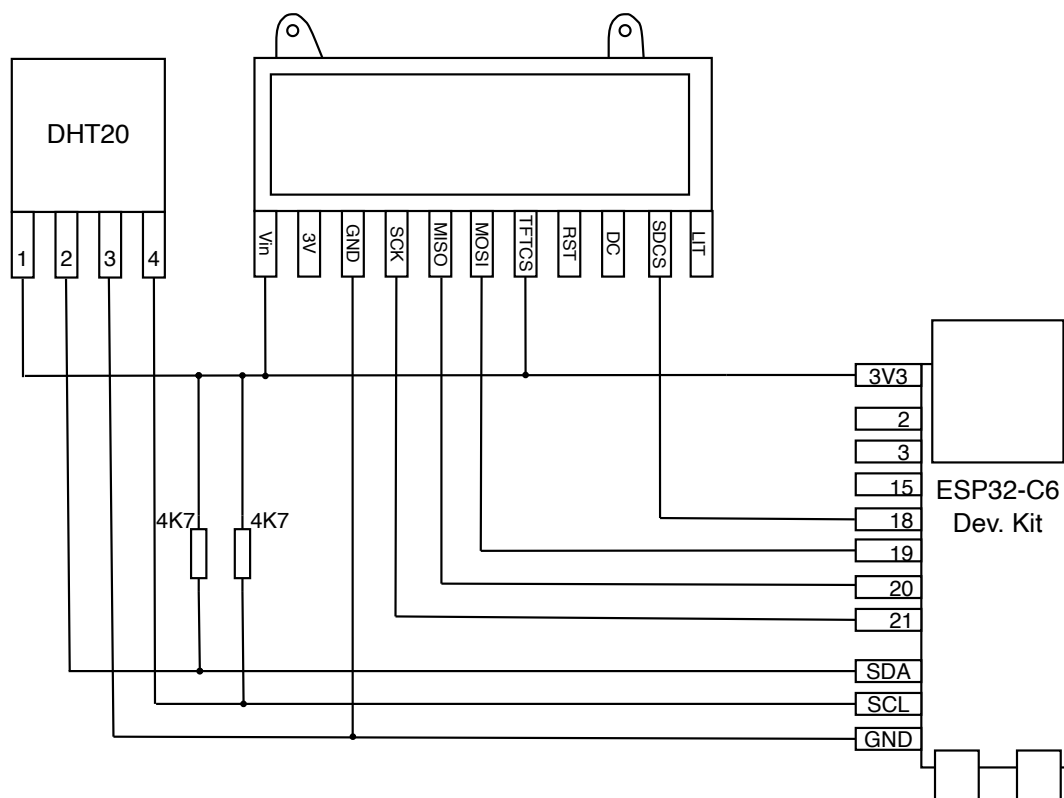


Figura 4: Circuito para integração do leitor de cartões com o sensor DHT20

A ficha técnica do DHT20 exige ainda um **condensador de desacoplamento de 100 nF entre VDD e GND**, colocado o mais próximo possível do sensor, e recomenda que as linhas SDA e SCL não corram paralelas e muito próximas, para evitar diafonia.

Resumo do protocolo do DHT20 (ficha técnica, secções 7 e 8):

- endereço I2C de 7 bits 0x38: o primeiro byte de cada transação é 0x70 para escrita e 0x71 para leitura; frequência de SCL até 400 kHz (*fast mode*).
- 1. após a alimentação, esperar pelo menos 100 ms e ler o byte de estado; se (`estado & 0x18`) != 0x18, inicializar os registos 0x1B, 0x1C e 0x1E (rotina disponibilizada pelo fabricante). Esta verificação só é necessária no arranque;
- 2. esperar 10 ms e enviar o comando 0xAC seguido dos parâmetros 0x33 e 0x00, que desencadeia uma medição;
- 3. esperar 80 ms e ler o estado: se o bit 7 for 1, o sensor ainda está ocupado e é preciso continuar a esperar;
- 4. ler 7 bytes: o estado (byte 0), cinco bytes de dados (bytes 1 a 5) e o CRC (byte 6), calculado como CRC8 com valor inicial 0xFF e polinómio $1 + x^4 + x^5 + x^8$;
- 5. o sinal de humidade S_{RH} (20 bits) ocupa os bytes 1 e 2 e o *nibble* superior do byte 3; o sinal de temperatura S_T (20 bits) ocupa o *nibble* inferior do byte 3 e os bytes 4 e 5. As grandezas obtêm-se com $RH = \frac{S_{RH}}{2^{20}} \times 100$ (em %RH) e $T = \frac{S_T}{2^{20}} \times 200 - 50$ (em °C).

Para limitar o autoaquecimento do sensor a menos de 0,1 °C, a ficha técnica (secção 4.4) indica que o tempo de ativação não deve exceder 10% do período de medição e recomenda uma medição a cada 2 s.

3.5.1 Comunicação entre tarefas com *queues* FreeRTOS

Na Questão 5, a aquisição de dados e a escrita no cartão são feitas por tarefas distintas. A razão é temporal: a leitura do sensor deve ocorrer com um período rigoroso, enquanto a duração de cada escrita no cartão é variável, porque abrir, escrever e fechar o ficheiro obriga a atualizar a tabela FAT e os diretórios. Se a mesma tarefa fizesse as duas coisas, qualquer escrita mais demorada atrasaria a amostra seguinte. Separando-as, a tarefa do sensor (produtora) mantém o seu período e a tarefa de *logging* (consumidora) escreve ao seu ritmo.

A ligação entre as duas é feita por uma *queue*: um *buffer* FIFO gerido pelo núcleo do FreeRTOS, com capacidade para um número fixo de itens, todos do mesmo tamanho. As funções essenciais são as seguintes ([FreeRTOS: xQueueSend](#), [xQueueReceive](#)):

- `xQueueCreate(uxQueueLength, uxItemSize)` cria a *queue* com capacidade para `uxQueueLength` itens de `uxItemSize` bytes e devolve o seu identificador, ou NULL se não houver memória disponível. Deve ser criada antes das tarefas que a usam;
- `xQueueSend(xQueue, &item, xTicksToWait)` coloca **uma cópia** do item no fim da *queue*. Os itens são copiados por valor, e não guardados por referência, pelo que a variável de origem pode ser reutilizada logo após o envio. Se a *queue* estiver cheia, a tarefa fica bloqueada no máximo `xTicksToWait ticks` à espera de espaço; com `xTicksToWait = 0`, a função retorna de imediato. Devolve `pdTRUE` em caso de sucesso ou `errQUEUE_FULL` se não conseguiu colocar o item;
- `xQueueReceive(xQueue, &item, xTicksToWait)` copia para `item` o elemento mais antigo e retira-o da *queue*. Se a *queue* estiver vazia, a tarefa passa ao estado *Blocked*, **sem consumir tempo de CPU**, até chegar um item ou se esgotar `xTicksToWait`. Devolve `pdTRUE` se recebeu um item ou `pdFALSE` se o tempo de espera terminou sem dados. Com `portMAX_DELAY`, a espera é indefinida.

A *queue* serve, assim, para transferir dados e, simultaneamente, para **sincronizar** as tarefas: a chegada de um item desbloqueia a tarefa consumidora, que não precisa de verificar periodicamente

se há dados (espera ativa). A capacidade da *queue* determina quanto atraso da consumidora é tolerado sem perdas: com uma amostra por segundo e capacidade para N itens, a tarefa de *logging* pode atrasar-se cerca de N segundos antes de a *queue* encher. O tempo de espera em *ticks* obtém-se a partir de milissegundos com a macro `pdMS_TO_TICKS()`.

Por exemplo, a estrutura trocada entre as tarefas pode ser definida assim:

```
typedef struct {
    int64_t    millis;
    uint32_t   counter;
    float      temperature;
    float      humidity;
} sample_t;

QueueHandle_t q = xQueueCreate(10, sizeof(sample_t));
```

QUESTÃO 5: Desenvolva um programa que, a cada segundo, leia a temperatura e a humidade do sensor DHT20 e registe esses valores no ficheiro `/sdcard/dht20.csv`. O programa deve ser organizado em **duas tarefas FreeRTOS que comunicam através de uma *queue***:

- a *queue* é criada em `app_main()`, antes das tarefas, com capacidade para 10 itens do tipo `sample_t`; a tarefa do sensor tem prioridade superior à da tarefa de *logging*;
- **tarefa do sensor:** com um período de 1000 ms garantido com `vTaskDelayUntil()`, obtém o tempo atual (`esp_timer_get_time() / 1000`), desencadeia uma medição no DHT20, lê o resultado e envia para a *queue* uma estrutura `sample_t` com os campos `millis`, `counter`, `temperature` e `humidity`. O contador é incrementado a **cada tentativa** de medição; se a leitura falhar (erro de comunicação I2C, sensor ocupado ou CRC inválido), é impresso um aviso com `ESP_LOGW()` e nada é enviado para a *queue*, pelo que as falhas ficam visíveis como lacunas no contador. O envio usa `xQueueSend()` com tempo de espera 0: se a *queue* estiver cheia, a função retorna `errQUEUE_FULL`, a amostra é descartada e é impresso um aviso, sem que o período de amostragem seja afetado;
- **tarefa de *logging*:** chama `xQueueReceive()` com um tempo de espera de 100 ms (`pdMS_TO_TICKS(100)`). Quando recebe uma estrutura, acrescenta uma linha ao ficheiro (abrindo-o e fechando-o a cada registo) e imprime na consola `Logged: time=<millis> ms, counter=<n>, temp=<t> C, hum=<h> %`. Quando a função retorna `pdFALSE` (tempo esgotado sem dados), verifica o estado do botão `BOOT`;
- formato do ficheiro: cabeçalho `millis,counter,temp_C,hum_pct`; temperatura e humidade com duas casas decimais; sem espaços; mesmas regras de criação/acrescento do ficheiro que na Questão 3;
- ao detetar o botão `BOOT` premido, a tarefa de *logging* termina o *logging* e desmonta o cartão, tal como na Questão 3;
- pode reutilizar o código de acesso ao DHT20 desenvolvido na aula anterior, mas deve usar o *driver* `driver/i2c_master.h`: o *driver* legado `driver/i2c.h` está em fim de vida desde o ESP-IDF v6.0 e será removido na v7.0. Recorde que cada medição demora cerca de 80 ms e que, após a alimentação, o sensor necessita de pelo menos 100 ms antes da primeira medição.

Teste o seu programa na placa, devendo obter um resultado semelhante ao seguinte na consola:

```
I (348) sdlog: Mounted. Card: 00000
```

```
I (350) sdlog: I2C initialized successfully for DHT20
I (352) main_task: Returned from app_main()
I (452) sdlog: Logged: time=362 ms, counter=0, temp=23.71 C, hum=74.96 %
I (1452) sdlog: Logged: time=1362 ms, counter=1, temp=23.72 C, hum=74.94 %
I (2452) sdlog: Logged: time=2362 ms, counter=2, temp=23.73 C, hum=74.91 %
I (3452) sdlog: Logged: time=3362 ms, counter=3, temp=23.73 C, hum=74.86 %
I (4452) sdlog: Logged: time=4362 ms, counter=4, temp=23.71 C, hum=74.91 %
I (5452) sdlog: Logged: time=5362 ms, counter=5, temp=23.71 C, hum=74.90 %
```

QUESTÃO 6: Termine o *logging* com o botão `BOOT`, desligue a placa e remova o cartão microSD.

- Consulte o conteúdo do ficheiro `dht20.csv` e compare-o com a informação impressa na consola. Existem lacunas no contador? Se sim, a que se devem?
- O programa abre e fecha o ficheiro a cada segundo, o que obriga a atualizar a tabela FAT e os diretórios do cartão em cada registo. Discuta o compromisso entre o risco de perda de dados e o desgaste do cartão (número limitado de ciclos de escrita) e proponha uma estratégia alternativa (p. ex. acumular N registos em memória antes de escrever). Que dados se perderiam, no pior caso, com a sua estratégia?
- Verifique se o período de medição de 1 s cumpre o critério de autoaquecimento da ficha técnica do DHT20 (tempo de ativação não superior a 10% do período). Que efeito teria na temperatura medida um período muito mais curto?
- Suponha que, ocasionalmente, uma escrita no cartão demora 4 s. Com a *queue* de 10 itens especificada, perdem-se amostras? A partir de que duração de escrita começariam a perder-se? Justifique com base no funcionamento de `xQueueSend()` com tempo de espera 0.

O ficheiro deverá ter um conteúdo semelhante ao seguinte (correspondente à execução apresentada acima):

```
millis,counter,temp_C,hum_pct
362,0,23.71,74.96
1362,1,23.72,74.94
2362,2,23.73,74.91
3362,3,23.73,74.86
4362,4,23.71,74.91
5362,5,23.71,74.90
```

3.6 Exercício opcional: observação dos sinais SPI

Se dispuser de um analisador lógico ou osciloscópio, ligue-o aos sinais `sck`, `mosi`, `miso` e `sdcs` e capture o arranque do programa da Questão 3. Identifique o comando `cmd0` e meça a frequência de `sck` durante a inicialização e durante as escritas no ficheiro. Os valores obtidos são consistentes com a sua resposta à alínea e) da Questão 1?

4 Resolução de problemas

Sintoma / mensagem	Causa provável	Ação
<code>Failed to initialize bus.</code>	GPIO inválidos ou barramento SPI já inicializado	Verificar a configuração dos pinos no SDK Configuration Editor; garantir que <code>spi_bus_initialize()</code> só é chamada uma vez
<code>Failed to initialize the card (ESP_ERR_TIMEOUT)</code>	O cartão não responde: cablagem, alimentação, <i>pull-ups</i> em falta, cartão mal inserido ou TFCs não ligado a 3,3 V	Verificar todas as ligações face à Tabela 3; reinserir o cartão
<code>ESP_ERR_INVALID_RESPONSE</code> OU <code>ESP_ERR_INVALID_CRC</code>	Sinais degradados (fios longos, mau contacto) ou MISO/MOSI trocados	Encurtar os fios; confirmar MISO/MOSI; reduzir a frequência (<code>host.max_freq_khz</code>)
<code>Failed to mount filesystem</code>	Cartão sem sistema de ficheiros FAT (p. ex. exFAT ou NTFS)	Formatar o cartão em FAT32 no portátil (após cópia de segurança)
<code>fopen()</code> devolve NULL	Caminho sem o prefixo <code>/sdcard</code> , nome longo com o suporte LFN desativado ou demasiados ficheiros abertos (<code>max_files</code>)	Verificar o caminho e o nome; consultar <code>strerror(errno)</code>
Ficheiro vazio ou incompleto no portátil	Alimentação retirada com o ficheiro aberto	Fechar o ficheiro após cada registo e desmontar o cartão antes de o remover

Sintoma / mensagem	Causa provável	Ação
Erro I2C na comunicação com o DHT20	Cablagem, <i>pull-ups</i> em falta, GPIO SDA/SCL trocados ou condensador de desacoplamento em falta	Verificar ligações; confirmar o endereço 0x38; reduzir a frequência de SCL

5 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)